

无人机多谱系纳管系统 — 新增业务模块开发文档

文档基本信息

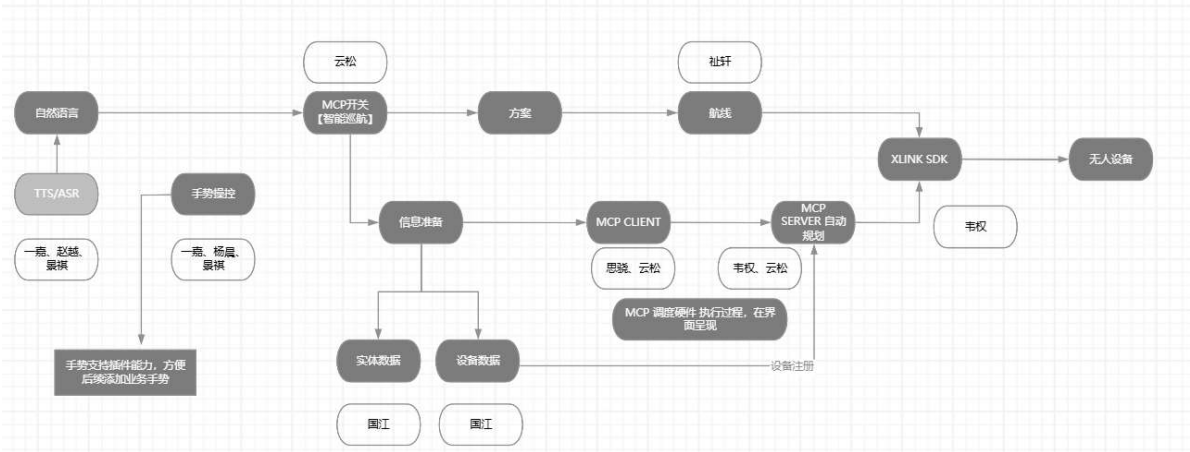
项目	内容
版本号	v1.0
编制日期	2026-05-25
方案来源	(多模态交互)、(MCP控制服务)、(Agent纳管 + 协议SDK)
目标读者	产品经理、前后端工程师、AI工程师、嵌入式/协议工程师

模块总览

模块编号	模块名称	方案来源	优先级	依赖关系
M-01	双向多模态交互模块	赵一嘉	P1	AI问答
M-02	无人装备控制MCP服务	李昊伟	P0	依赖M-03、任务管理
M-03	异构协议标准化SDK	田耀鹏	P0	被M-02依赖

模块依赖关系：

用户交互层：	M-01（多模态交互） ↓ 指令解析
执行控制层：	M-02（MCP控制服务） ↓ 协议调用
设备适配层：	M-03（协议标准化SDK） ↓ 无人机设备端



一、双向多模态交互模块（M-01）

1.1 功能概述

多模态交互模块是系统的新型人机接口层，打破传统键盘鼠标输入范式，支持**语音、手势等多通道指令输入**，同时由平台按事件分级主动向用户推送状态与告警（TTS语音、视觉告警），构建双向闭环**的人机交互体系。

该模块的核心价值在于两点：一是在高强度、双手被占用的操控场景中降低操作负担；二是在紧急事件发生时，确保告警信息能以不可忽视的方式触达操作员。

双向交互架构：

打开语音输入-》语音输入-》语音确认发送-》开启手势识别-》操作地图-》关闭手势识别



1.2 核心功能逻辑

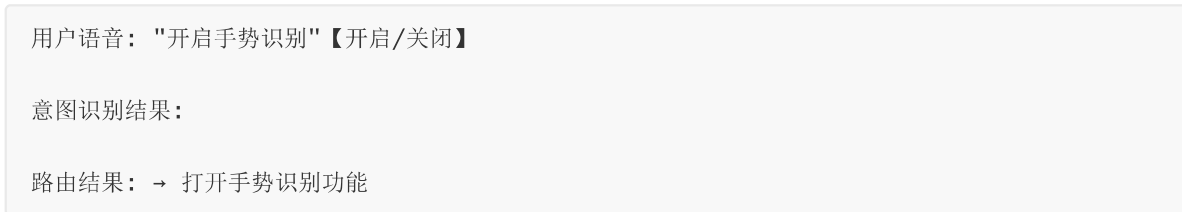
1.2.1 多模态输入处理

增加一个手势识别意图, 陈越提供

语音输入链路：



意图分类与槽位示例：



手势识别链路：



预定义手势映射表：

可以视能力匹配【放大、缩小、拖拉】-》后续再匹配 业务功能

手势名称	手势描述	映射指令
拖拉	大拇指+食指捏合拖动	鼠标按住左键拖拉
放大	握拳松开成手掌	鼠标滚轮滚动放大功能
缩小	五指捏合	鼠标滚轮滚动缩小功能

1.2.2 指令安全分级

后续所有多模态输入的指令用于执行业务操作前设定安全分级校验。【暂时不做】

1.3 核心数据模型

无

1.4 交互逻辑

语音指令完整处理时序：

1. 用户唤醒词激活（"小巡"）

2. VAD 检测到语音开始

3. ASR 实时转写（流式，150ms延迟内）

4. VAD 检测到语音结束

5. NLU 解析意图

6. 安全等级判断：
 执行，TTS简短回复"好的"

7. 执行对应功能，TTS播报结果

1.5 接口清单

接口	方法	路径	描述
处理多模态输入	POST	/api/multimodal/input	接收前端采集的多模态原始数据
TTS播报	POST	/api/multimodal/tts	主动触发TTS播报

1.6 与其他模块的交互

输出能力：

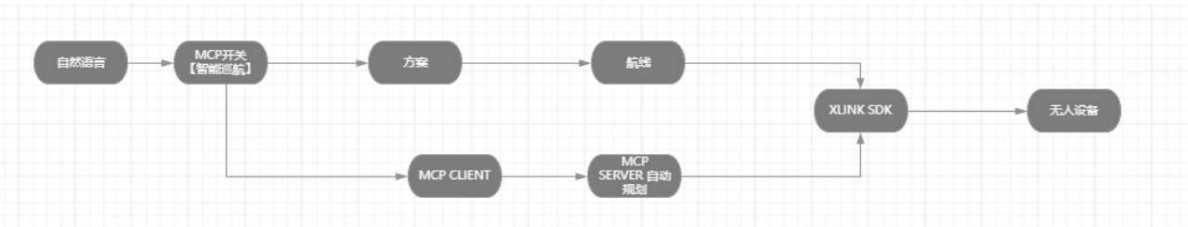
目标模块	提供能力
AI问答	作为语音输入通道，识别结果送入AI问答

二、无人装备控制MCP服务（M-02）

2.1 功能概述

MCP（Model Context Protocol）控制服务是**方案到设备执行**的关键桥接层，解决现有系统中"任务方案到任务执行"过渡不精确、缺乏执行状态反馈的核心问题。

核心设计思想：LLM不直接操控无人机协议，而是生成语义级任务方案；方案经人工审核后进入任务管理模块，形成驻留任务；每个驻留任务对应一个**MCP Client**，与**MCP Server**保持持续通信，维护完整的任务状态机，直至任务终态。



文档：<https://arxiv.org/html/2601.15486v1>
MCP SERVER:<https://github.com/PeterJBurke/droneserver/>

代码要多熟悉一下，会涉及到跟我们的服务进行数据交互

io.modelcontextprotocol.sdk

mcp

0.1.1

org.springframework.ai

spring-ai-mcp-client-spring-boot-starter

2.0.0

2.2 系统角色与分层架构



角色	职责	实例数量
LLM / 任务规划端	生成语义级任务方案	—
任务管理模块	审核方案, 创建任务实例	—
MCP Client	每个任务对应一个Client, 维护状态机, 与MCP Server通信	N个 (N=并发任务数)
MCP Server	统一控制入口, 屏蔽底层协议, 管理无人机控制Client	1个主服务
无人机控制Client	由MCP Server内部启动, 控制具体无人机	M个 (M=在飞设备数)
无人机端Server	接收指令, 返回状态	每台设备1个

2.3 核心数据模型

2.3.1 MCP 任务数据模型

```
MCP任务 MCPTask {
  taskId:          MCP任务唯一ID
  missionId:       关联的任务管理模块任务ID
  sessionId:       MCP通信会话ID

  // 任务方案（LLM生成）
  taskPlan: {
    commands: 指令序列列表 [{
      commandId:  指令ID（顺序编号）
      commandType: 指令类型 枚举：TAKEOFF/GOTO/HOVER/LAND/RTH/CAPTURE/SCAN/CUSTOM
      params: {
        targetCoord: 目标坐标 {lat, lng, alt}（GOTO类）
        speed:       飞行速度 m/s
        altitudeMode: 高度模式 枚举：RELATIVE/ABSOLUTE
        duration:    持续时间(秒)（HOVER类）
        captureMode: 拍摄模式（CAPTURE类）
      }
      precondition: 前置条件表达式（上一指令状态=SUCCESS时才执行）
      timeout:      指令超时时间(秒)
    }]
    targetDevice: 执行设备ID
    priority:     任务优先级
  }

  // 状态机
  currentState:  当前状态（见状态枚举）
  currentCommandIdx: 当前执行到第几条指令
  stateHistory: 状态变更历史 [{
    fromState: 原状态
    toState:   新状态
    timestamp: 变更时间
    reason:    变更原因
    operator:  操作人（人工干预时记录）
  }]

  // 运行时数据
  deviceSnapshot: {
    timestamp: 快照时间
    position:  {lat, lng, alt}
    speed:     当前速度
    battery:   电量
    linkQuality: 链路质量
    status:    飞行状态
  }

  // 异常处置
  anomalyContext: 异常上下文（ACTIVE_SAMPLING状态时填充）
  recoveryAction: 选择的恢复动作

  createTime:  创建时间
  completedTime: 完成时间
}
```

2.3.2 任务状态机定义



各状态详细说明：

状态	类型	触发条件	可执行操作
INIT	过程态	任务实例创建, MCP Client启动	等待资源就绪
READY	过程态	设备在线、电量充足、协议连接成功	开始执行第一条指令
IN_PROGRESS	过程态	指令正在执行, 设备上报正常心跳	暂停/终止/监控
ACTIVE_SAMPLING	异常处理态	MCP Server检测到异常或不确定状态, 需要上层决策	提供处置选项供操作员/LLM选择
RECOVERING	异常处理态	决策确认后, 执行重连/返航/绕行等恢复动作	监控恢复进度
INTERRUPTED	异常终态	恢复失败, 或异常确认后无法继续按原计划	触发事件通知, 生成事故报告
SUCCESS	成功终态	所有指令顺序执行完毕, 设备安全落地	生成完成报告
FAILED	失败终态	系统错误、协议不可恢复、设备硬件故障	触发工单
ABORTED	主动终止终态	操作员主动终止	触发RTH保护
CLOSED	闭合终态	Session关闭, 设备控制Client释放	—

2.4 核心功能逻辑

2.4.1 任务执行主流程

以"机动-拍照-降落"为例：

1. 任务方案

```
plan.commands = [
  {commandType: GOTO, params: {coord: A}},
  {commandType: CAPTURE},
  {commandType: LAND}
]
```

2. 任务管理模块创建MCP任务，状态→INIT

3. MCP Client向MCP Server发起Session建立请求

携带：taskId + deviceId + plan

4. MCP Server检查资源：

- 设备DJI-001是否在线（心跳 < 5秒）
- 电量是否足够完成所有指令（估算耗电 < 可用电量×0.8）
- 目标坐标是否在禁飞区内

检查通过 → 状态→READY

5. MCP Server启动无人机控制Client，与DJI-001建立设备级连接

6. 开始执行 commandIdx=0（GOTO）

- 状态→IN_PROGRESS
- 无人机端Server持续上报位置（1Hz）
- MCP Server将状态封装后推送MCP Client
- MCP Client更新 deviceSnapshot

7. 设备到达目标坐标A

- commandIdx=0 执行完成（precondition检查通过）
- 执行 commandIdx=1（CAPTURE）
- 触发拍照，等待拍照完成回调

8. 拍照成功

- commandIdx=1 完成
- 执行 commandIdx=2（LAND）

9. 设备降落并上报"已降落"状态

- 所有指令完成
- 状态→SUCCESS→CLOSED

10. MCP Client向任务管理模块回写完成状态

- 任务状态更新为"已完成"
- 触发飞行记录生成、报告系统更新

2.5 MCP通信协议设计

Session建立请求：

```
MCPSessionRequest {
  taskId:      MCP任务ID
  deviceId:    目标设备ID
  taskPlan:    任务方案（命令序列）
  sessionConfig: {
    heartbeatInterval: 心跳间隔(秒) 默认1
    statusPushInterval: 状态推送间隔(秒) 默认2
    commandTimeoutSec: 单条指令超时时间(秒) 默认60
    anomalyThresholds: 异常检测阈值配置
  }
}
```

MCP Server → MCP Client 状态推送:

```
MCPStatusPush {
  sessionId:    会话ID
  taskId:      任务ID
  pushType:    推送类型 枚举: HEARTBEAT/STATE_CHANGE/COMMAND_COMPLETE/ANOMALY

  taskState:    当前任务状态
  commandIdx:   当前执行指令序号
  deviceSnapshot: 设备当前状态快照

  // anomaly类型时附加
  anomalyDetail: {
    anomalyType: 异常类型
    anomalyDesc: 异常描述
    suggestedActions: 建议处置列表
    autoResolveIn: 自动执行默认动作的倒计时(秒)
  }

  timestamp:    推送时间
}
```

2.6 接口清单

接口	方法	路径	描述
创建MCP任务	POST	/api/mcp/tasks	任务方案审核后创建
获取MCP任务状态	GET	/api/mcp/tasks/{taskId}	查询任务状态机
任务控制操作	POST	/api/mcp/tasks/{taskId}/control	暂停/恢复/终止
异常处置决策	POST	/api/mcp/tasks/{taskId}/anomaly-resolve	提交处置决策
MCP状态订阅	WebSocket	/ws/mcp/tasks/{taskId}	实时状态推送
活跃任务列表	GET	/api/mcp/tasks/active	所有进行中任务
历史任务查询	GET	/api/mcp/tasks/history	支持分页和状态筛选
任务执行日志	GET	/api/mcp/tasks/{taskId}/logs	完整执行轨迹

2.7 与其他模块的交互

输入依赖：

来源模块	获取内容	用途
任务管理模块	审核后的任务方案（命令序列）	MCP任务创建输入
协议标准化SDK（M-03）	设备控制底层接口	发送实际飞行指令
标注管理模块	禁飞区坐标	指令执行前合规校验
PHM模块	设备健康状态	任务前置资源检查
Counter-UAS模块	空域威胁告警	触发ACTIVE_SAMPLING

输出能力：

目标模块	提供能力
任务管理模块	任务执行状态回写（完成/中断/失败）
飞行记录模块	完整飞行轨迹、设备状态时序
多模态交互（M-01）	任务执行中的P0/P1状态告警
报告系统	任务执行数据供报告生成
Agent纳管（）	任务状态供Agent监控和动态调整

三、异构协议标准化SDK集成模块（M-03）

3.1 功能概述

异构协议标准化SDK模块是整个系统的**设备连接基础设施**，解决现有系统中不同无人机厂商（DJI、Autel、自研飞控等）协议差异导致的二次开发成本高、接入困难等问题。

设计思想：通过**适配器模式（Adapter Pattern）**，将各种异构协议统一封装为标准SDK接口，上层模块（MCP服务、任务管理、实时监控等）只调用标准接口，无需关心底层协议细节。

解决的核心问题：

现有问题	SDK模块的解决方案
DJI/Autel/自研飞控协议各异	统一标准接口屏蔽差异
新接入一款设备需大量二次开发	只需实现协议Adapter，无需改动上层
控制指令格式不统一	标准指令Schema，Adapter负责转换
状态数据格式不统一	统一设备状态模型，Adapter负责映射
连接管理分散在各模块	SDK统一管理连接生命周期

3.2 架构设计



↓

DJI设备
(御/经纬)

↓

开源飞控
(PX4/APM)

3.3 标准SDK接口定义

3.3.1 标准指令集 (统一Command Schema)

```
标准控制指令 StandardCommand {
  taskId: 任务ID
  commandId: 指令唯一ID
  commandType: 指令类型 枚举 (见下表)
  deviceId: 目标设备ID
  adapterType: 代理协议类型 枚举

  params: { // 按commandType动态填充
    // GOTO指令
    targetLat: 目标纬度
    targetLng: 目标经度
    targetAlt: 目标高度(m)
    speed: 飞行速度(m/s)
    altitudeMode: 高度模式 枚举: RELATIVE/ABSOLUTE/AGL

    // HOVER指令
    duration: 悬停时长(秒), -1=无限悬停

    // TAKEOFF指令
    takeoffAlt: 起飞高度(m)

    // CAPTURE指令
    captureMode: 枚举: PHOTO/VIDEO_START/VIDEO_STOP/TIMELAPSE

    // GIMBAL指令
    pitch: 俯仰角
    roll: 横滚角
    yaw: 偏航角

    // CUSTOM指令 (厂商特有指令透传)
    rawPayload: 原始指令数据(base64)
    adapterHint: 指定处理的Adapter类型
  }

  timeout: 执行超时时间(秒)
  priority: 指令优先级
}
```

标准指令类型枚举:

飞行控制: TAKEOFF, LAND, RTH, GOTO, HOVER, ORBIT, WAYPOINT_MISSION

视角控制: GIMBAL_CONTROL, ZOOM, SWITCH_CAMERA

媒体控制: CAPTURE_PHOTO, START_RECORD, STOP_RECORD

系统控制: LOCK, UNLOCK, REBOOT, SET_MODE

透传指令: CUSTOM (用于厂商特有功能)

3.4 Adapter 接口规范

每个协议Adapter必须实现以下标准接口：

Adapter接口规范（实现契约）：

```
interface DeviceAdapter {

    // 1. 连接管理
    connect(deviceConfig)      → ConnectionResult
    disconnect(deviceId)      → void
    getConnectionStatus()     → ConnectionStatus

    // 2. 指令发送（将标准指令转换为协议指令并发送）
    sendCommand(StandardCommand) → CommandResult {
        success:    boolean
        commandId:  指令ID
        message:    结果描述
        nativeCode: 协议原始返回码
    }

    // 3. 状态获取
    getStatus(deviceId)        → StandardDeviceStatus

    // 4. 状态订阅（推送模式）
    subscribe(deviceId, callback) → SubscriptionHandle
    unsubscribe(handle)          → void

    // 5. Adapter元信息
    getSupportedCommandTypes() → CommandType[]
    getAdapterInfo()           → AdapterInfo {
        adapterCode:    适配器编码
        supportedModels: 支持的设备型号列表
        protocolVersion: 协议版本
        maxConcurrentDevices: 最大并发设备数
    }
}
```

3.4.1 已实现Adapter清单

Adapter编码	协议类型	支持设备	特殊能力
DJI_ADAPTER	DJI MSDK/OSDK	Mavic 3E, Matrice 300 RTK 等	精准RTK定位、多载荷 控制
MAVLINK_ADAPTER	MAVLink v2	PX4/ArduPilot飞控平台	开源生态、高度可定制
SIMULATOR_ADAPTER	模拟器协议	软件在环仿真(SITL)	开发测试用，无实际设 备
CUSTOM_ADAPTER	用户自定义	用户开发的适配器	插件化热加载

3.5 插件化Adapter扩展机制

为支持未来新型无人机协议，Xlink SDK提供插件化扩展机制[后续实现]：

自定义Adapter接入流程：

1. 开发者实现 `xlinkAdapter` 接口

2. 打包为JAR/动态库

3. 放入 `adapters/` 插件目录

4. 调用注册接口：
POST `/api/sdk/adapters/register`
{
 adapterCode: "MY_ADAPTER",
 adapterClass: "com.example.MyAdapter",
 jarPath: "/adapters/my-adapter.jar"
}

5. SDK动态加载，立即可用

6. 在设备配置中指定 `adapterCode: "MY_ADAPTER"`

3.6 接口清单

接口	方法	路径	描述
连接设备	POST	<code>/api/sdk/devices/{deviceId}/connect</code>	建立设备连接
断开设备	DELETE	<code>/api/sdk/devices/{deviceId}/connect</code>	断开连接
获取连接状态	GET	<code>/api/sdk/devices/{deviceId}/connection</code>	查询连接状态
发送标准指令	POST	<code>/api/sdk/devices/{deviceId}/commands</code>	发送控制指令
查询设备状态	GET	<code>/api/sdk/devices/{deviceId}/status</code>	当前标准状态
设备状态订阅	WebSocket	<code>/ws/sdk/devices/{deviceId}</code>	实时状态推送
更新设备配置	PUT	<code>/api/sdk/devices/{deviceId}/config</code>	更新连接参数
设备健康诊断	GET	<code>/api/sdk/devices/{deviceId}/diagnostic</code>	连接质量诊断
指令执行历史	GET	<code>/api/sdk/devices/{deviceId}/command-history</code>	查询历史指令
原始数据透传	POST	<code>/api/sdk/devices/{deviceId}/raw</code>	CUSTOM 指令透传

3.7 与其他模块的交互

被依赖关系（作为基础设施）：

调用方模块	使用SDK的方式
MCP控制服务（M-02）	发送飞行控制指令、订阅设备状态
实时监控模块	订阅遥测数据流，驱动前端展示
设备管理模块	建立/断开设备连接，获取设备状态
PHM规则引擎	订阅遥测数据流，规则评估输入

输出能力：

能力	描述
统一连接管理	所有设备连接由SDK统一管理，上层无需关心重连逻辑
标准状态推送	统一格式的设备状态，1Hz推送频率
指令执行反馈	同步返回指令接受结果，异步回调执行完成状态
多协议透明支持	上层代码切换设备型号无需修改

四、模块间整体协作流程

4.1 完整任务执行端到端流程（四模块联动）

场景：操作员语音下达"搜救任务，在A区搜寻失踪人员"

Step 1：多模态输入（M-01）

· ASR识别："搜救任务，在A区搜寻失踪人员"

· NLU解析：`intent=agent.goal.submit, scene=SEARCH_RESCUE, area=A区`

· 安全等级：Level 2（需语音确认）

· TTS播报："将启动搜救Agent执行A区搜寻任务，请确认"

· 操作员语音确认

Step 2：Agent纳管规划（ ）

· 纳管平台分配 `SEARCH_RESCUE_AGENT` 实例

· Agent调用设备管理Skill：获取附近可用设备

· Agent调用地图Skill：获取A区地图和禁飞区

· LLM生成搜救方案：

– 阶段1：DJI-001 覆盖北半区（S型扫描航线）

– 阶段2：DJI-002 覆盖南半区

– 阶段3：发现目标后追踪并等待指令

· 方案展示给操作员确认（关键决策人在环）

· 操作员确认后批量创建任务

Step 3：MCP执行控制（M-02）

· 任务管理模块创建2个MCP任务

· MCP Client 1 管理DJI-001任务（GOTOxN + CAPTURExN）

· MCP Client 2 管理DJI-002任务

· 各MCP Client向MCP Server建立Session

· MCP Server向协议SDK申请设备控制权

Step 4: 协议SDK执行 (M-03)

- SDK DJI_ADAPTER 连接DJI-001、DJI-002
- 将标准GOTO/CAPTURE指令转换为DJI OSDK指令发送
- 持续接收设备心跳和位置，转换为标准状态推送给MCP Server

Step 5: 任务执行中的联动

- MCP Server持续将状态推送给MCP Client
- Agent持续监控任务进度
- AI事件模块检测到疑似人员目标
- M-01推送P1告警: TTS"发现疑似目标, 坐标N40.2° E116.4°"
- Agent动态调整: DJI-001悬停追踪目标, DJI-002继续覆盖
- 操作员确认目标身份 → Agent发送跟踪指令

Step 6: 任务完成

- 搜救目标确认找到
- MCP任务状态→SUCCESS
- Agent目标状态→COMPLETED
- M-01推送P2告警: TTS"搜救任务完成, 目标已确认"
- 报告系统生成搜救任务报告

五、验收标准

5.1 M-01 多模态交互

- 语音识别准确率 > 95% (标准普通话, 安静环境)
- 意图识别准确率 > 90% (针对系统内置意图体系)
- 手势识别6种预定义手势, 准确率 > 92%
- P0级事件推送端到端延迟 < 500ms
- Level 3危险指令必须双模态确认才执行
- 环境噪声>65dB时自动切换为手势主输入
- TTS播报内容与告警事件信息一致

5.2 M-02 MCP控制服务

- 任务状态机流转完整覆盖所有定义状态
- ACTIVE_SAMPLING 在异常检测后1秒内触发
- 超时60秒无响应后自动执行RTH默认动作
- MCP Session建立成功率 > 99% (设备在线情况下)
- 设备状态推送延迟 < 500ms
- 多指令串行执行不出现时序错误
- 每个任务对应唯一MCP Client, 资源不混用

5.3M-03 协议标准化SDK

- 新Adapter接入无需修改上层任何代码
- 标准状态模型覆盖所有必要字段, 缺失字段填充默认值
- 插件化Adapter热加载, 无需重启服务
- 连接断开后自动重连, 重连成功率 > 98% (设备恢复在线情况下)
- 指令发送延迟 < 100ms (本地网络)
- SIMULATOR_ADAPTER支持开发测试, 行为与真实设备一致

六、关键数据表清单

表名	所属模块	用途
multimodal_input_event	M-01	多模态输入事件记录
platform_push_event	M-01	平台推送事件记录
gesture_config	M-01	手势映射配置
event_level_config	M-01	事件分级配置
mcp_task	M-02	MCP任务和状态机
mcp_command_log	M-02	指令执行日志
mcp_anomaly_record	M-02	异常处置记录
ai_agent		Agent注册信息
agent_goal		Agent目标和规划
agent_replan_history		Agent重规划历史
agent_resource_conflict		资源冲突记录
device_adapter	M-03	Adapter注册信息
device_connection_config	M-03	设备连接配置
sdk_command_history	M-03	SDK指令历史

文档编制: 产品与技术架构团队
参考方案: 多模态交互、MCP控制服务、协议SDK
技术审核: AI工程师 / 后端架构师